

Investigación: Redes Programables y el Lenguaje P4

Isaac Araya Quesada – C4C567
Carlos Alvarado Meneses – C4C337

29 de junio de 2026

1 Evolución del plano de datos

1.1 Limitaciones del modelo de red tradicional

El modelo de red tradicional se fundamenta en una arquitectura donde los dispositivos de interconexión de datos, tales como switches y routers, ejecutan sus funciones de procesamiento de paquetes mediante hardware especializado denominado Circuitos Integrados de Aplicación Específica (*Application-Specific Integrated Circuits*, ASICs). Estos ASICs de función fija son diseñados e implementados en silicio por el fabricante con un conjunto predefinido de protocolos (por ejemplo, IPv4, IPv6, MPLS).

Esta rigidez arquitectónica introduce limitaciones críticas en los entornos de red modernos:

- **Ciclos de innovación restrictivos y lentos:** Si surge la necesidad de desplegar un nuevo protocolo de encapsulamiento o de telemetría, el plano de datos existente no puede ser modificado. Esto obliga a la industria a depender de un ciclo de hardware que típicamente toma entre 3 y 5 años para el diseño, fabricación y distribución de nuevos chips físicos.
- **Dependencia absoluta del proveedor (*Vendor Lock-in*):** Los operadores de red carecen de autonomía para implementar optimizaciones propietarias en el plano de datos. Cualquier cambio o corrección de comportamiento en la lógica de procesamiento de paquetes depende de las actualizaciones de firmware y de las hojas de ruta comerciales de los fabricantes de hardware.

1.2 Surgimiento del concepto de plano de datos programable

Como respuesta a las limitaciones del silicio de función fija, la comunidad académica y de ingeniería propuso el concepto de **plano de datos programable**. Este paradigma redefine la relación entre el software y el silicio de red, postulando que el hardware no debe dictar los protocolos de la red, sino actuar como un lienzo genérico que ejecuta instrucciones de procesamiento compiladas mediante software.

A diferencia de las CPUs genéricas, un plano de datos programable utiliza arquitecturas optimizadas para examinar, modificar y redirigir paquetes a velocidades de terabits por segundo, garantizando el procesamiento a nivel de línea (*line-rate*) sin importar la complejidad del protocolo definido por el usuario.

1.3 Comparación entre SDN clásica (OpenFlow) y P4

La llegada de las Redes Definidas por Software (SDN) supuso una separación inicial entre el plano de control y el plano de datos. Sin embargo, existen diferencias fundamentales entre la primera generación de SDN y el enfoque moderno de P4:

- **SDN Clásica (OpenFlow):** OpenFlow basa su arquitectura en la manipulación remota de un **plano de datos que sigue siendo fijo** [2]. El estándar OpenFlow define explícitamente un conjunto de cabeceras predeterminadas que el switch

puede reconocer. El plano de control puede poblar reglas en estas tablas mediante operaciones de coincidencia y acción (*match-action*), pero si un operador requiere procesar un campo personalizado o un protocolo emergente no estandarizado, OpenFlow resulta incapaz de modificar la estructura interna del hardware.

- **Redes Programables con P4:** P4 subvierte este límite al ser completamente **independiente de los protocolos** (*Protocol-Independent*) [1]. P4 no asume la existencia previa de IPv4 o Ethernet. En su lugar, proporciona las abstracciones de software necesarias para que el programador defina con precisión milimétrica la estructura de las cabeceras, la lógica de los extractores de datos y el flujo condicional de las tablas en el hardware de red[cite: 1].

2 El lenguaje P4

2.1 Historia y motivación

El lenguaje P4 (*Programming Protocol-Independent Packet Processors*) fue presentado formalmente en el año 2014 a través del artículo fundacional publicado por Bosshart et al. titulado "*P4: Programming Protocol-Independent Packet Processors*" [1].

La motivación subyacente del proyecto consistió en democratizar el desarrollo de infraestructura de red, otorgando a los ingenieros de sistemas el mismo grado de flexibilidad y control expresivo sobre el silicio de red que los desarrolladores de gráficos tridimensionales obtuvieron históricamente mediante la introducción de las GPUs y lenguajes de sombreado programables.

2.2 Arquitectura de un programa P4

De acuerdo con la especificación estándar del lenguaje [3], un programa P4.16 se compone de abstracciones modulares fundamentales que estructuran el ciclo de vida del paquete dentro del dispositivo:

- **Parsers (Analizadores Sintácticos):** Son máquinas de estados finitos encargadas de examinar el flujo lineal de bytes entrantes desde la interfaz física. Su objetivo es identificar y extraer campos de cabecera estructurados, almacenándolos en representaciones de memoria interna para su posterior procesamiento[cite: 1].
- **Match-Action Tables (Tablas de Coincidencia y Acción):** Constituyen el núcleo lógico del procesamiento. Estas tablas especifican qué campos (por ejemplo, la dirección IP de destino) se utilizarán como llaves de búsqueda y qué tipos de coincidencia se aplicarán (coincidencia exacta, por prefijo más largo *LPM*, o ternaria). Al encontrar una coincidencia, ejecutan una función o acción asociada (como reescribir una dirección MAC o decretar un descarte)[cite: 1].

- **Control Flows (Flujos de Control):** Actúan como la función principal que coordina el pipeline. Determinan de forma imperativa y condicional la secuencia exacta en la que se invocan las tablas de coincidencia y acción sobre un paquete[cite: 1].
- **Deparsers (Desanalizadores Sintácticos):** Realizan la operación inversa al *parser*. Toman las estructuras de cabecera modificadas por la lógica del programa, las serializan de nuevo en un flujo continuo de bytes junto con la carga útil (*payload*) original del paquete y lo preparan para su transmisión hacia la interfaz de salida[cite: 1].

2.3 Modelo de ejecución PISA

La abstracción de hardware en la cual se fundamenta el diseño de P4 se denomina **Arquitectura de Switch Independiente del Protocolo** (*Protocol Independent Switch Architecture*, PISA). El modelo PISA organiza el procesamiento de paquetes de manera estrictamente lineal y segmentada a través de un pipeline estructurado de la siguiente forma:

Parser $\longrightarrow$ Ingress Match-Action $\longrightarrow$ Traffic Manager (Queue) $\longrightarrow$ Egress Match-Action $\longrightarrow$ Deparser

Cuando un paquete ingresa, el *Parser* extrae sus cabeceras. Luego, el bloque *Ingress Match-Action* procesa el paquete para tomar decisiones lógicas globales como la determinación de la interfaz de salida. El paquete pasa al *Traffic Manager*, donde se gestionan las colas de prioridad y el almacenamiento intermedio. Posteriormente, el bloque *Egress Match-Action* ejecuta transformaciones de última milla específicas de la interfaz de salida (como la encapsulación final de túneles) antes de que el *Deparser* reconstruya el paquete físico.

2.4 Tipos de Targets

El código escrito en P4 es agnóstico al hardware final y requiere ser compilado mediante el compilador universal **p4c** hacia objetivos de ejecución específicos denominados *targets* [1]:

- **BMv2 (Behavioral Model v2):** Es una implementación de software de referencia que emula el comportamiento exacto de un switch P4 en entornos virtuales de CPU [6]. Se utiliza ampliamente en contextos de aprendizaje, desarrollo ágil y simulaciones topológicas en redes Mininet, aunque carece de optimizaciones para rendimiento en producción[cite: 1].
- **Intel Tofino:** Representa la encarnación comercial en silicio de la arquitectura PISA. Es un ASIC nativamente programable capaz de procesar flujos de datos masivos a escalas de múltiples Terabits por segundo sin introducir penalizaciones de latencia con respecto a los switches tradicionales.

- **FPGAs (Field Programmable Gate Arrays):** Dispositivos basados en bloques de lógica reprogramable en hardware que ofrecen un punto medio idóneo entre el rendimiento de procesamiento en paralelo cercano al silicio puro y la flexibilidad de reconfiguración parcial mediante software.
-

3 P4Runtime y el control plane

3.1 Rol del plano de control en una arquitectura P4

Mientras que la especificación y compilación del código P4 determina de manera unívoca la estructura estática del pipeline (las cabeceras, los analizadores y la existencia de las tablas), el plano de datos resultante permanece pasivo en términos de políticas de red. El plano de control (*Control Plane*) asume la responsabilidad de gestionar la inteligencia de la red en tiempo de ejecución[cite: 26]. Esto incluye calcular rutas, ejecutar protocolos de enrutamiento dinámico (como BGP o OSPF), implementar políticas de seguridad y, fundamentalmente, poblar con datos reales las llaves y acciones de las tablas de *match-action* del switch.

3.2 Protocolo P4Runtime y comunicación vía gRPC

P4Runtime es un framework de control unificado y de código abierto que actúa como la interfaz API estándar para la comunicación bidireccional entre el plano de control y el plano de datos[cite: 25]. A diferencia de los enfoques tradicionales de red basados en sockets crudos, P4Runtime utiliza **gRPC** como transporte de red y *Protocol Buffers* (protobuf) para la serialización estructurada de los mensajes[cite: 25].

La innovación disruptiva de P4Runtime radica en su naturaleza dinámica: durante la compilación del programa P4 mediante **p4c**, se genera automáticamente un archivo de metadatos estandarizado llamado **p4info**. Este archivo describe la arquitectura exacta del plano de datos (qué tablas existen, qué campos esperan y qué acciones exponen). P4Runtime lee este archivo en tiempo de ejecución, lo que permite al controlador interactuar con el switch utilizando una API con tipado estricto construida a la medida del programa P4, eliminando la necesidad de que el switch tenga protocolos grabados en piedra.

3.3 Comparación con OpenFlow: similitudes y diferencias

Para comprender el salto evolutivo que representa P4Runtime, es imperativo analizarlo en contraste con OpenFlow, el protocolo pionero de la primera ola de las Redes Definidas por Software (SDN)[cite: 20, 27]:

- **Similitudes:**

- **Separación de planos:** Ambos paradigmas se fundamentan en el principio SDN de desacoplar de forma estricta el plano de control (lógica y toma de decisiones) del plano de datos (reenvío de paquetes).
- **Control centralizado de tablas:** Tanto OpenFlow como P4Runtime permiten que un controlador lógicamente centralizado y remoto configure, inserte, actualice y elimine reglas dentro de las tablas de coincidencia y acción (*match-action tables*) alojadas en el dispositivo físico.
- **Mecanismos de telemetría asíncrona:** Ambos frameworks proveen canales de comunicación asíncronos para reportar eventos desde el switch hacia el controlador, tales como la expiración de flujos por inactividad o notificaciones de paquetes que no coincidieron con ninguna regla (*packet-in*).

■ **Diferencias:**

- **Independencia del protocolo frente a dependencia intrínseca:** OpenFlow está ligado de forma nativa a un conjunto rígido de protocolos predefinidos (v1.0 reconocía 12 campos fijos, extendidos en versiones posteriores). Si un campo no está en la especificación de OpenFlow, el controlador no puede usarlo. P4Runtime es completamente agnóstico al protocolo (*Protocol-Independent*); manipula cualquier cabecera o campo arbitrario que haya sido modelado en el programa P4[cite: 23].
- **API Dinámica vs. API Fija:** La API de OpenFlow está congelada en el silicio y definida estrictamente por la versión del estándar que implemente el switch. Por el contrario, la API de P4Runtime se genera de manera dinámica y bajo demanda en tiempo de compilación a través del archivo `p4info`. La API cambia y se adapta cada vez que el desarrollador modifica el código fuente de su switch.
- **Control de Población vs. Control de Arquitectura:** OpenFlow solo permite configurar *el contenido* de las tablas dentro de un pipeline cuyo diseño estructural es fijo e inmutable. P4Runtime, al trabajar en conjunto con P4, otorga control tanto sobre *la arquitectura* del pipeline (creación de tablas y lógica de parseo) como sobre *el contenido* (reglas de reenvío), redefiniendo las capacidades del silicio por completo.
- **Tecnología de serialización:** OpenFlow utiliza un formato binario propietario y rígido sobre conexiones TCP/TLS directas, lo que dificulta su extensión. P4Runtime aprovecha tecnologías modernas de la industria de software como gRPC y Protocol Buffers, facilitando la integración nativa con sistemas distribuidos, optimizando la serialización y permitiendo la autogeneración de bibliotecas de control en múltiples lenguajes (Python, Go, C++).

4 Casos de uso relevantes

4.1 In-Band Network Telemetry (INT)

La Telemetría de Red En Banda (INT) representa uno de los hitos tecnológicos más importantes habilitados por las redes programables [4]. Bajo los esquemas tradicionales (como SNMP o NetFlow), la recolección de métricas internas de los dispositivos se realiza mediante muestreos periódicos que saturan el plano de control.

INT aprovecha la flexibilidad de P4 para permitir que los propios switches inserten metadatos de diagnóstico directamente dentro de las cabeceras de los paquetes de datos de los usuarios a medida que estos atraviesan los pipelines de la red [4]. Cada switch intermedio puede añadir de manera transparente información sobre el identificador único del switch visitado, las marcas de tiempo exactas de entrada y salida (*timestamps*) para el cálculo microscópico de latencias por salto, y el estado de utilización de las colas de memoria. Al llegar al extremo final de la red, un switch colector remueve estas cabeceras de telemetría y envía los datos agregados a un sistema de analítica, logrando una visibilidad en tiempo real a nivel de paquete sin generar tráfico de control adicional.

4.2 Load Balancing (Balanceo de Carga) Programable

El balanceo de carga en los centros de datos masivos tradicionalmente requiere de servidores o appliances dedicados de Capa 4, lo que introduce latencias adicionales y puntos únicos de fallo. Mediante el uso de P4, es posible programar algoritmos avanzados de balanceo de carga directamente dentro del silicio de conmutación de la red. Los switches programables pueden calcular funciones Hash consistentes basadas en los flujos de tráfico entrantes para distribuir las conexiones uniformemente a través de un grupo de servidores en nanosegundos, manejando millones de conexiones simultáneas a nivel de línea y reaccionando de inmediato ante la caída de un servidor físico.

4.3 Funciones de seguridad en el data plane

Tradicionalmente, las funciones avanzadas de seguridad (como los Firewalls de Inspección de Estados o los sistemas de mitigación de ataques) se relegan a *appliances* de software externos debido a que los ASICs fijos no pueden alterar su comportamiento lógicamente para rastrear estados dinámicos. P4 transforma este panorama al permitir la ejecución de lógica de seguridad directamente en el plano de datos a velocidades de silicio (*line-rate*):

- **Mitigación de ataques de Denegación de Servicio (DDoS):** Mediante el uso de estructuras de datos primitivas en P4 como los *Registers*, el plano de datos puede almacenar contadores, calcular tasas de llegada de paquetes por IP de origen y ejecutar algoritmos de mitigación (como *Token Bucket*) en nanosegundos. Si el volumen de paquetes supera un umbral crítico, el switch descarta el tráfico malicioso inmediatamente antes de que sature las colas de procesamiento o los servidores internos.

- **Firewalls con inspección de estado (*Stateful Firewalls*):** P4 permite rastrear el ciclo de vida de los flujos de transporte (por ejemplo, el apretón de manos de tres vías de TCP o *TCP 3-way handshake*) directamente en el hardware mediante máquinas de estado implementadas en registros locales. De este modo, el switch puede bloquear paquetes de respuesta TCP de manera autónoma si no existe un registro previo de que la conexión fue iniciada legítimamente desde el interior de la red.
- **Filtrado dinámico y empaquetado seguro:** Se pueden codificar reglas complejas basadas en campos personalizados de las cabeceras para aislar el tráfico de red, detectar anomalías estructurales en los paquetes (malformaciones) y aplicar políticas de control de acceso (*ACLs*) altamente granulares sin degradar el ancho de banda del dispositivo.

4.4 Casos de uso en entornos de producción reales

- **Meta (Facebook):** Ha rediseñado parte de su infraestructura de centros de datos desplegando switches basados en P4 para contrarrestar ataques masivos de DDoS. Al programar algoritmos de mitigación y filtrado directamente en el plano de datos, logran identificar firmas de tráfico malicioso y descartar paquetes atacantes en el primer punto de contacto de la red, protegiendo las capas internas de servidores web con latencias mínimas.
- **Google:** Emplea abstracciones de planos de datos programables en sus sistemas de red definidos por software de gran escala (*Andromeda*). Esto les otorga la capacidad de desplegar parches de seguridad virtuales, reconfigurar encapsulamientos de red en la nube de forma dinámica y aislar el tráfico de los inquilinos (*tenants*) con un control granular que los ASICs de función fija tradicionales no permiten implementar sin reemplazar la infraestructura física.

5 Referencias

Referencias

- [1] Bosshart, P., Daly, D., Gibb, G., Izzard, M., McKeown, N., Rexford, J., ... & Walker, D. (2014). P4: Programming protocol-independent packet processors. *ACM SIGCOMM Computer Communication Review*, 44(3), 87-95.
- [2] McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., ... & Turner, J. (2008). OpenFlow: enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review*, 38(2), 69-74.
- [3] P4 Language Consortium. (2023). *P4_16 Language Specification version 1.2.4*. Disponible en: <https://p4.org/p4-spec/>

- [4] Kim, C., Sivaraman, A., Katta, N., Bas, A., Dixit, A., & Wobker, L. (2015). In-band network telemetry (INT). *P4.org Technical Report*.
- [5] P4.org API Working Group. (2023). *P4Runtime Specification version 1.3.0*. Disponible en: <https://p4.org/p4runtime/>
- [6] P4 Language Consortium. (2024). *Behavioral Model v2 (BMv2) Reference Switch Architecture*. Disponible en: <https://github.com/p4lang/behavioral-model>
- [7] Alcoz, S. V., Dietzel, C., & Vanbever, L. (2020). Tracking, Tuning, and Terminating Elephant Flows in Programmable Data Planes. *IEEE/ACM Transactions on Networking*, 28(4), 1410-1423.
- [8] Shrivastav, V. (2019). Fast, scalable, and programmable packet processing in software. *Proceedings of the ACM SIGCOMM 2019 Conference*, 320-334.